

SecureBank black box test design

Author: Aldo Ramon Velazquez Fonseca (A4Id0) Assignment: Homework 4, Module 4 Scope: required assignment plus the dual-language bonus only.

Specification and explicit assumptions

Tests derive from the assignment's public banking rules. Python and JavaScript will implement the same public contract. `design/cases.json` will contain the concrete inputs, expected results and checkpoints for every implemented case. Its IDs link the design to both suites. No test will inspect private fields.

Type	Minimum balance	Monthly fee	Waiver (strictly greater than)	Daily transfer limit
Savings	100.00	5.00	1000.00	2000.00
Checking	0.00	10.00	5000.00	5000.00
Premium	10000.00	0.00	Always free	50000.00

The assignment leaves several details open. The following are requirements for this simulation, not claims about a production banking service:

1. Money is numeric, finite, nonnegative for initial balances and strictly positive for transactions, with at most two decimal places. Booleans and numeric strings are invalid. Store integer cents; reject amounts or resulting balances above 1,000,000,000.00 (a shared safe arithmetic cap).
2. Opening below the minimum is permitted but starts Suspended. Equality to the minimum is Active. Deposits restoring **at least** the minimum reactivate an account. This resolves the wording "above minimum" against "below minimum".
3. Only Active accounts can transfer/pay bills. Suspended accounts can receive deposits; Frozen and Closed accounts reject monetary operations. Freeze is allowed only from Active. Unfreeze is allowed only from Frozen. Closed is terminal. Closing twice is rejected. Balance/history remain readable.
4. Fee waivers use `>` (not `>=`), following the account-type descriptions. Process fees once per month on the first day, for Active/Suspended accounts. Insufficient funds suspend the account without overdrawing or recording a debit. That month's attempt is consumed; no same-month retry charge.
5. An injected UTC date clock makes midnight and month rollover deterministic. The cumulative transfer allowance resets on the next UTC date, even for an idle account. Bill payments do not consume the transfer allowance.
6. Bill payees are the registered IDs `utilities`, `credit-card`, and `internet`. Dates must be real, zero-padded ISO dates. Schedule only today or later; validate available funds at scheduling and again at execution. Scheduling does not reserve money. Due payments execute once in insertion order; failed execution is terminal and recorded in the returned batch result. Processing the queue is an administrative action permitted while Frozen, but every attempted payment fails. Closed accounts reject queue processing entirely.

7. Transfers simulate the source-account debit and record a nonempty destination label (external by default). There is no real settlement or multi-account atomicity. Account information contains a nonempty owner name only.

8. History uses inclusive dates; reversed/invalid ranges are errors. CSV has date, kind, amount, balance, detail columns, two-decimal money, quoted details, doubled embedded quotes and LF line endings. Returned records are copies.

9. Error precedence is state, amount, destination/payee/date, funds, daily limit. Rejected transactions leave balances, allowance and history unchanged.

1. Equivalence partitioning (EP)

Transfer amount

IDs	Class	Valid?	Representative	Expected
EP01	Positive, affordable, under limit	Yes	500	Debit 500
EP02	Zero	No	0	Amount must be positive
EP03	Negative	No	-100	Amount must be positive
EP04	Above allowance	No	5001 with 10000 balance	Exceeds daily limit
EP05	Above available funds	No	501 with 500 balance	Insufficient funds
EP06-EP08	String, boolean, sub-cent	No	"10", true, 0.001	Invalid amount

Account type

IDs	Class	Valid?	Representative	Expected
EP09-EP11	Supported type	Yes	Savings, Checking, Premium	Correct minimum and daily limit
EP12-EP13	Unknown or missing type	No	Business, null	Invalid account type

Opening balance

IDs	Class	Valid?	Representative	Expected
EP14	Nonnegative below minimum	Yes	Savings 50	Suspended
EP15	At/above minimum	Yes	Savings 200	Active
EP16-EP18	Negative, string, sub-cent	No	-1, "100", 0.001	Invalid opening balance

Payee information

IDs	Class	Valid?	Representative	Expected
EP19	Registered ID	Yes	utilities, 20	Payment recorded
EP20-EP22	Unknown, empty, missing	No	stranger, "", null	Invalid payee

History date range

IDs	Class	Valid?	Representative	Expected
EP23	Ordered interval containing activity	Yes	2026-09-01 to 2026-09-30	Matching entries
EP24	Ordered interval without activity	Yes	2026-08-01 to 2026-08-31	Empty list
EP25	Reversed interval	No	Sep 30 to Sep 1	Invalid date range
EP26-EP28	Bad format, impossible date, missing	No	09/22/2026, 2026-02-30, null	Invalid date

Additional EP cases cover destinations, owner updates, scheduling, CSV escaping, copy isolation, non-finite numbers and the explicit amount cap. Their exact oracles are listed in the case catalog.

2. Boundary value analysis (BV)

Unless stated otherwise, accounts are Active, have enough funds, and have used none of their daily allowance. Money values are in dollars.

IDs	Boundary	Values in test order	Expected in same order
BV01-BV06	Checking transfer [0.01, 5000]	0, 0.01, 4999.99, 5000, 5000.01, 10000	reject, accept, accept, accept, reject, reject
BV07-BV12	Savings transfer [0.01, 2000]	0, 0.01, 1999.99, 2000, 2000.01, 4000	reject, accept, accept, accept, reject, reject
BV13-BV18	Savings minimum 100	0, 99.98, 99.99, 100, 100.01, 200	Suspended x3, Active x3
BV19-BV24	Savings waiver >1000	999.98, 999.99, 1000, 1000.01, 1000.02, 2000	fee 5 x3, fee 0 x3
BV25-BV30	Checking waiver >5000	4999.98, 4999.99, 5000, 5000.01, 5000.02, 10000	fee 10 x3, fee 0 x3
BV31-BV36	Remaining daily allowance 100 after 4900	0, 0.01, 99.99, 100, 100.01, 200	reject, accept, accept, accept, reject, reject
BV37-BV42	Available funds 100	0, 0.01, 99.99, 100, 100.01, 101	reject, accept, accept, accept, reject, reject
BV43-BV48	Premium minimum 10000	0, 9999.98, 9999.99, 10000, 10000.01, 20000	Suspended x3, Active x3

Additional boundaries: UTC midnight reset, inclusive history endpoints, today versus past/future payments, valid leap day, exact deposit restoration, and first/last days around monthly fee processing. See the case catalog.

3. Decision tables (DT)

Transfer validation (positive amount, valid destination)

Condition/action	DT01	DT02	DT03	DT04	DT05	DT06	DT07	DT08
Active?	Y	Y	Y	Y	N	N	N	N
Sufficient funds?	Y	Y	N	N	Y	Y	N	N
Within remaining allowance?	Y	N	Y	N	Y	N	Y	N
Result	Success	Limit error	Funds error	Funds error	State error	State error	State error	State error

For combinations with a used allowance, first transfer 4900 from a Checking account. Then attempt 50 or 200 with remaining funds 25 or 1000. Non-Active combinations freeze through the public API. Suspended and Closed are tested by the state suite. Only the first applicable error is returned.

Monthly fee processing (first day, no prior attempt)

Rule	Type	Balance	Waived?	Can afford fee?	Fee / resulting state
DT09	Savings	1000.01	Y	Y	0 / Active
DT10	Savings	1000	N	Y	5 / Active
DT11	Savings	100	N	Y	5 / Suspended
DT12	Savings	5	N	Y	5 / Suspended
DT13	Savings	4.99	N	N	no debit / Suspended, error
DT14	Checking	5000.01	Y	Y	0 / Active
DT15	Checking	5000	N	Y	10 / Active
DT16	Checking	10	N	Y	10 / Active
DT17	Checking	9.99	N	N	no debit / Suspended, error
DT18	Premium	10000	Y	n/a	0 / Active
DT19	Premium	9999	Y	n/a	0 / Suspended

Outer guards: Frozen/Closed reject before fee calculation; non-first days and repeat attempts return success with zero charged. Waived and failed attempts are also marked processed. These are covered in the supplemental catalog.

Bill payment validation (positive amount, today's date)

Condition/action	DT20	DT21	DT22	DT23	DT24	DT25	DT26	DT27
Active?	Y	Y	Y	Y	N	N	N	N
Registered payee?	Y	Y	N	N	Y	Y	N	N
Sufficient funds?	Y	N	Y	N	Y	N	Y	N
Result	Success	Funds error	Payee error	Payee error	State error	State error	State error	State error

4. State transition testing (ST)

Diagram described as directed edges (the required base diagram; no visual bonus): Active -> Suspended on debit below minimum or unaffordable fee; Suspended -> Active on a deposit reaching the minimum; Active -> Frozen on a customer/fraud freeze; Frozen -> Active on approved unfreeze; Active, Suspended, Frozen -> Closed on close; Closed -> Closed for every rejected mutation.

Current state	Event	Next state	Output
Active	Debit leaves balance below minimum	Suspended	warning=true
Active	Valid debit/deposit retaining minimum	Active	success
Active	Unaffordable fee	Suspended	insufficient fee funds
Active	Freeze	Frozen	transactions blocked
Suspended	Deposit reaches minimum	Active	warning=false
Suspended	Smaller deposit	Suspended	warning=true
Suspended	Transfer/payment/freeze/unfreeze	Suspended	error, no mutation
Frozen	Approved unfreeze	Active	operations restored
Frozen	Debit/deposit/fee/freeze	Frozen	error, no mutation
Any non-Closed	Close	Closed	final balance returned
Closed	Any mutation including repeated close	Closed	account closed
Any	Read balance/history	Same	view-only result

Test	Start	Event/sequence	Expected checkpoint
ST01	Active Savings 100	transfer 0.01	Suspended, 99.99, warning
ST02	Suspended Savings 99	deposit 1	Active, exactly 100
ST03	Suspended Savings 50	deposit 49	Suspended, 99

Test	Start	Event/sequence	Expected checkpoint
ST04	Active	freeze, transfer, deposit, bill, fee	Frozen; mutations rejected
ST05	Active	freeze, unfreeze, transfer	Active; debit succeeds
ST06	Active	close, transfer, deposit, unfreeze, freeze, update, fee, bill, close	Closed; no reopening
ST07	Suspended	close	Closed
ST08	Frozen	close	Closed
ST09	Suspended	transfer, bill, freeze, unfreeze	unchanged; rejected
ST10	Active	unfreeze	unchanged; invalid transition
ST11	Active Checking 9	fee on first	Suspended, balance unchanged
ST12	Active Savings 100	fee, deposit 5	Suspended then Active

Execution and coverage plan

Implement independent Python/pytest and JavaScript/Jest versions, sharing only JSON scenarios, not production code. Four test files per language load EP, BV, DT, and ST cases. Each scenario constructs a fresh account and fake clock. Assert exact operation results plus declared public snapshots. Error operations also assert unchanged snapshots unless fee suspension is explicitly expected. Constructor validation checks the exact exception message. Copy-isolation cases modify a returned history entry and verify the account remains unchanged.

Run both suites with branch coverage, requiring **over 80% line and branch coverage** in each implementation. Record actual metrics, logs and screenshots; never infer passing results from code inspection. Measure each technique alone to explain overlaps and gaps. The numeric catalog is a shared human-written oracle; equivalence does not establish correctness if that oracle is wrong.

No CI workflow or professional visual diagrams are included. Publishing, opening the PR, and Canvas submission remain the student's responsibility.